Group Topic : Arithmetic Expression Evaluator

Group Name :

1. **Moaz**
2. **Hasnain**
3. **Saad**
4. **Zohaib**
5. **Faizan**
6. **Abdulrahan**
7. **Bilal**

1. Problem Statement

Arithmetic expressions in programming and mathematics include operators such as addition (+), subtraction (-), multiplication (*), and division (/). Correct evaluation of these expressions requires proper handling of operator precedence, associativity, and parentheses.

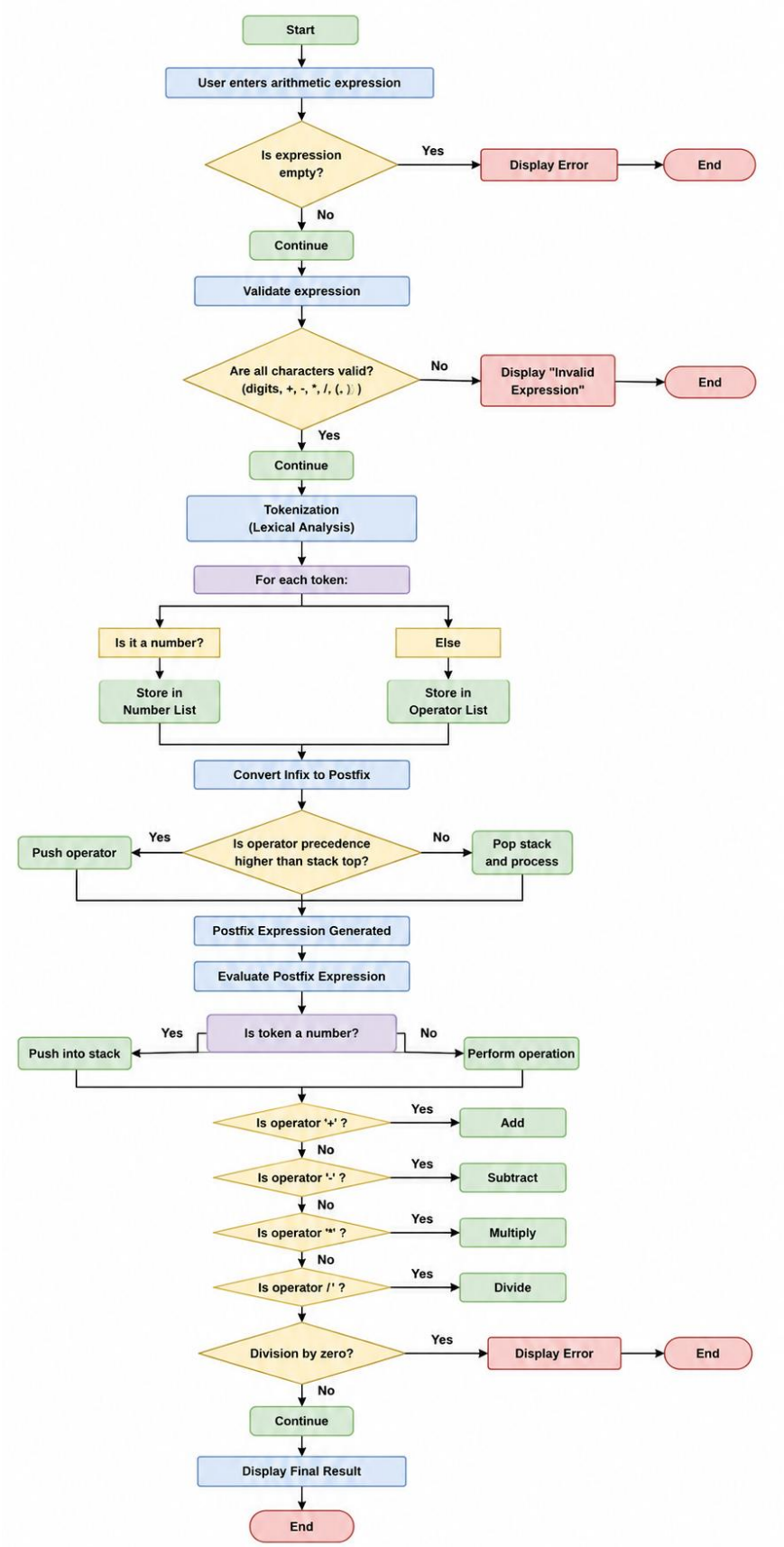
Manually evaluating complex expressions can lead to errors and inefficiency. Therefore, there is a need to design a system that can automatically process and evaluate arithmetic expressions in a structured and reliable manner.

The objective of this project is to develop an application that:

- Accepts an arithmetic expression from the user
- Validates the correctness of the expression
- Converts the expression from infix notation to postfix notation
- Evaluates the postfix expression to compute the final result
- Handles errors such as invalid syntax, unbalanced parentheses, and division by zero

This project implements core concepts of Compiler Construction, including lexical analysis, syntax validation, stack-based processing, and expression evaluation.

WorkFlow :


Start
User enters arithmetic expression
Is expression empty?
Yes
Display Error
End
No
Continue
Validate expression
Are all characters valid? (digits, +, -, *, /, (,))
No
Display "Invalid Expression"
End
Yes
Continue
Tokenization (Lexical Analysis)
For each token:
Is it a number?
Else
Store in Number List
Store in Operator List
Convert Infix to Postfix
Is operator precedence higher than stack top?
Yes
Push operator
No
Pop stack and process
Postfix Expression Generated
Evaluate Postfix Expression
Is token a number?
Yes
Push into stack
No
Perform operation
Is operator '+' ?
Yes
Add
No
Is operator '-' ?
Yes
Subtract
No
Is operator '*' ?
Yes
Multiply
No
Is operator /' ?
Yes
Divide
Division by zero?
Yes
Display Error
End
No
Continue
Display Final Result
End

3. System Design

Class 1: ExpressionInput

Purpose:

Handles user input and basic validation of arithmetic expressions.

Object:

- inputObj

Functions:

- getExpression() → Takes expression from user
- validateInput() → Checks if input is empty or contains invalid characters

---

Class 2: Tokenizer (Lexical Analyzer)

Purpose:

Breaks the input expression into meaningful tokens (numbers and operators).

Object:

- tokenObj

Functions:

- tokenize() → Splits expression into tokens

Example:

Input: 12+5*3
Output: 12 , + , 5 , * , 3

Class 3: Validator

Purpose:

Ensures the expression follows correct arithmetic rules.

Object:

- validatorObj

Functions:

- checkParentheses() → Checks balanced brackets
- checkOperators() → Detects invalid operator sequences

Example:

Invalid: 5++6

Class 4: InfixToPostfixConverter

Purpose:

Converts infix expression into postfix notation using stack.

Object:

- converterObj

Functions:

- convert() → Converts infix to postfix
- precedence() → Returns operator priority

Example:

Infix: 2+3*4
Postfix: 2 3 4 * +

Class 5: StackManager

Purpose:

Implements stack operations used in conversion and evaluation.

Object:

- stackObj

Functions:

- push() → Insert element
- pop() → Remove top element
- peek() → View top element
- isEmpty() → Check stack status

Class 6: PostfixEvaluator

Purpose:

Evaluates postfix expression to produce final result.

Object:

- evaluatorObj

Functions:

- evaluate() → Computes final result
- performOperation() → Executes +, -, *, / operations

Class 7: ResultManager

Purpose:

Handles output display and error reporting.

Object:

- resultObj

Functions:

- displayResult() → Shows final output
- displayError() → Displays error messages

Conclusion

This project demonstrates a structured approach to arithmetic expression evaluation using Compiler Construction concepts. By dividing the system into modular components and applying stack-based techniques, the project efficiently processes expressions from input to final evaluation. It also highlights real-world compiler processes such as lexical analysis, parsing, and expression evaluation.